

1.1 节练习

练习 1.1: 查阅你使用的编译器的文档，确定它所使用的文件命名约定。编译并运行第 2 页的 main 程序。

练习 1.2: 改写程序，让它返回-1。返回值-1 通常被当作程序错误的标识。重新编译并运行你的程序，观察你的系统如何处理 main 返回的错误标识。

1.2 初识输入输出

C++语言并未定义任何输入输出（IO）语句，取而代之，包含了一个全面的**标准库**（standard library）来提供 IO 机制（以及很多其他设施）。对于很多用途，包括本书中的示例来说，我们只需了解 IO 库中一部分基本概念和操作。

本书中的很多示例都使用了 **iostream** 库。iostream 库包含两个基础类型 **istream** 和 **ostream**，分别表示输入流和输出流。一个流就是一个字符序列，是从 IO 设备读出或写入 IO 设备的。术语“流”（stream）想要表达的是，随着时间的推移，字符是顺序生成或消耗的。

标准输入输出对象

6

标准库定义了 4 个 IO 对象。为了处理输入，我们使用一个名为 **cin**（发音为 see-in）的 istream 类型的对象。这个对象也被称为**标准输入**（standard input）。对于输出，我们使用一个名为 **cout**（发音为 see-out）的 ostream 类型的对象。此对象也被称为**标准输出**（standard output）。标准库还定义了两个 ostream 对象，名为 **cerr** 和 **clog**（发音分别为 see-err 和 see-log）。我们通常用 cerr 来输出警告和错误消息，因此它也被称为**标准错误**（standard error）。而 clog 用来输出程序运行时的一般性信息。

系统通常将程序所运行的窗口与这些对象关联起来。因此，当我们读取 cin，数据将从程序正在运行的窗口读入，当我们向 cout、cerr 和 clog 写入数据时，将会写到同一个窗口。

一个使用 IO 库的程序

在书店程序中，我们需要将多条记录合并成单一的汇总记录。作为一个相关的，但更简单的问题，我们先来看一下如何将两个数相加。通过使用 IO 库，我们可以扩展 main 程序，使之能提示用户输入两个数，然后输出它们的和：

```
#include <iostream>
int main()
{
    std::cout << "Enter two numbers:" << std::endl;
    int v1 = 0, v2 = 0;
    std::cin >> v1 >> v2;
    std::cout << "The sum of " << v1 << " and " << v2
              << " is " << v1 + v2 << std::endl;
    return 0;
}
```

这个程序开始时在用户屏幕打印

Enter two numbers:

然后等待用户输入。如果用户键入

3 7

然后键入一个回车，则程序产生如下输出：

The sum of 3 and 7 is 10

程序的第一行

```
#include <iostream>
```

告诉编译器我们想要使用 `iostream` 库。尖括号中的名字（本例中是 `iostream`）指出了一个头文件（header）。每个使用标准库设施的程序都必须包含相关的头文件。`#include` 指令和头文件的名字必须写在同一行中。通常情况下，`#include` 指令必须出现在所有函数之外。我们一般将一个程序的所有 `#include` 指令都放在源文件的开始位置。

向流写入数据

`main` 的函数体的第一条语句执行了一个表达式（expression）。在 C++ 中，一个表达式产生一个计算结果，它由一个或多个运算对象和（通常是）一个运算符组成。这条语句中的表达式使用了输出运算符（`<<`）在标准输出上打印消息：

```
std::cout << "Enter two numbers:" << std::endl;
```

`<<` 运算符接受两个运算对象：左侧的运算对象必须是一个 `ostream` 对象，右侧的运算对象是要打印的值。此运算符将给定的值写到给定的 `ostream` 对象中。输出运算符的计算结果就是其左侧运算对象。即，计算结果就是我们写入给定值的那个 `ostream` 对象。

我们的输出语句使用了两次 `<<` 运算符。因此此运算符返回其左侧的运算对象，因此第一个运算符的结果成为了第二个运算符的左侧运算对象。这样，我们就可以将输出请求连接起来。因此，我们的表达式等价于

```
(std::cout << "Enter two numbers:") << std::endl;
```

链中每个运算符的左侧运算对象都是相同的，在本例中是 `std::cout`。我们也可以用两条语句生成相同的输出：

```
std::cout << "Enter two numbers:";  
std::cout << std::endl;
```

第一个输出运算符给用户打印一条消息。这个消息是一个字符串字面值常量（string literal），是用一对双引号包围的字符序列。在双引号之间的文本被打印到标准输出。

第二个运算符打印 `endl`，这是一个被称为操纵符（manipulator）的特殊值。写入 `endl` 的效果是结束当前行，并将与设备关联的缓冲区（buffer）中的内容刷到设备中。缓冲刷新操作可以保证到目前为止程序所产生的所有输出都真正写入输出流中，而不是仅停留在内存中等待写入流。



程序员常常在调试时添加打印语句。这类语句应该保证“一直”刷新流。否则，如果程序崩溃，输出可能还留在缓冲区中，从而导致关于程序崩溃位置的错误推断。

使用标准库中的名字

细心的读者可能会注意到这个程序使用了 `std::cout` 和 `std::endl`，而不是直接的 `cout` 和 `endl`。前缀 `std::` 指出名字 `cout` 和 `endl` 是定义在名为 **std** 的命名空间 (namespace) 中的。命名空间可以帮助我们避免不经意的名字定义冲突，以及使用库中相同名字导致的冲突。标准库定义的所有名字都在命名空间 `std` 中。



8

通过命名空间使用标准库有一个副作用：当使用标准库中的一个名字时，必须显式说明我们想使用来自命名空间 `std` 中的名字。例如，需要写出 `std::cout`，通过使用**作用域运算符** (`::`) 来指出我们想使用定义在命名空间 `std` 中的名字 `cout`。3.1 节 (第 74 页) 将给出一个更简单的访问标准库中名字的方法。

从流读取数据

在提示用户输入数据之后，接下来我们希望读入用户的输入。首先定义两个名为 `v1` 和 `v2` 的变量 (variable) 来保存输入：

```
int v1 = 0, v2 = 0;
```

我们将这两个变量定义为 `int` 类型，`int` 是一种内置类型，用来表示整数。还将它们初始化 (initialize) 为 0。初始化一个变量，就是在变量创建的同时为它赋予一个值。

下一条语句是

```
std::cin >> v1 >> v2;
```

它读入输入数据。**输入运算符** (`>>`) 与输出运算符类似，它接受一个 `istream` 作为其左侧运算对象，接受一个对象作为其右侧运算对象。它从给定的 `istream` 读入数据，并存入给定对象中。与输出运算符类似，输入运算符返回其左侧运算对象作为其计算结果。因此，此表达式等价于

```
(std::cin >> v1) >> v2;
```

由于此运算符返回其左侧运算对象，因此我们可以将一系列输入请求合并到单一语句中。本例中的输入操作从 `std::cin` 读入两个值，并将第一个值存入 `v1`，将第二个值存入 `v2`。换句话说，它与下面两条语句的执行结果是一样的

```
std::cin >> v1;  
std::cin >> v2;
```

完成程序

剩下的就是打印计算结果了：

```
std::cout << "The sum of " << v1 << " and " << v2  
          << " is " << v1 + v2 << std::endl;
```

这条语句虽然比提示用户输入的打印语句更长，但原理上是一样的，它将每个运算对象打印在标准输出上。本例一个有意思的地方在于，运算对象并不都是相同类型的值。某些运算对象是字符串面值常量，例如 `"The sum of "`。其他运算对象则是 `int` 值，如 `v1`、`v2` 以及算术表达式 `v1+v2` 的计算结果。标准库定义了不同版本的输入输出运算符，来处理这些不同类型的运算对象。



9

1.2 节练习

练习 1.3: 编写程序, 在标准输出上打印 Hello, World。

练习 1.4: 我们的程序使用加法运算符+来将两个数相加。编写程序使用乘法运算符*, 来打印两个数的积。

练习 1.5: 我们将所有输出操作放在一条很长的语句中。重写程序, 将每个运算对象的打印操作放在一条独立的语句中。

练习 1.6: 解释下面程序片段是否合法。

```
std::cout << "The sum of " << v1;
          << " and " << v2;
          << " is " << v1 + v2 << std::endl;
```

如果程序是合法的, 它输出什么? 如果程序不合法, 原因何在? 应该如何修正?

1.3 注释简介

在程序变得更复杂之前, 我们应该了解一下 C++ 是如何处理注释 (comments) 的。注释可以帮助人类读者理解程序。注释通常用于概述算法, 确定变量的用途, 或者解释晦涩难懂的代码段。编译器会忽略注释, 因此注释对程序的行为或性能不会有任何影响。

虽然编译器会忽略注释, 但读者并不会。即使系统文档的其他部分已经过时, 程序员也倾向于相信注释的内容是正确可信的。因此, 错误的注释比完全没有注释更糟糕, 因为它会误导读者。因此, 当你修改代码时, 不要忘记同时更新注释!

C++ 中注释的种类

C++ 中有两种注释: 单行注释和界定符对注释。单行注释以双斜线 (//) 开始, 以换行符结束。当前行双斜线右侧的所有内容都会被编译器忽略, 这种注释可以包含任何文本, 包括额外的双斜线。

另一种注释使用继承自 C 语言的两个界定符 (/* 和 */)。这种注释以 /* 开始, 以 */ 结束, 可以包含除 */ 外的任意内容, 包括换行符。编译器将落在 /* 和 */ 之间的所有内容都当作注释。

注释界定符可以放置于任何允许放置制表符、空格符或换行符的地方。注释界定符可以跨越程序中的多行, 但这并不是必须的。当注释界定符跨越多行时, 最好能显式指出其内部的程序行都属于多行注释的一部分。我们所采用的风格是, 注释内的每行都以一个星号开头, 从而指出整个范围都是多行注释的一部分。

10

程序中通常同时包含两种形式的注释。注释界定符对通常用于多行解释, 而双斜线注释常用于半行和单行附注。

```
#include <iostream>
/*
 * 简单主函数:
 * 读取两个数, 求它们的和
 */
int main()
{
```